

X Bot Detector

A Chrome extension that scores any X profile and tweet thread for bot likelihood in real time, backed by an XGBoost model trained on 37,438 labelled accounts.

Problem Statement

Automated accounts make up a large and growing share of activity on social media. The 2024 Imperva Bad Bot Report found that almost half of all internet traffic in 2023 was automated, with bad bots responsible for 32% of total traffic (Imperva, 2024). On X, automated accounts amplify low-credibility content (Shao et al., 2018), distort political discourse during elections (Bessi and Ferrara, 2016; Howard, Woolley and Calo, 2018), and increase exposure to negative material (Stella, Ferrara and De Domenico, 2018). As an active user on X, I often cannot tell what is real or automated (Ferrara et al., 2016; Cresci, 2020).

Existing bot detection tools either need academic API access, live on a separate website, or sit behind a paid subscription. None of them run inside the browser where the user already is. This project fills that gap by building a bot detection model that only uses profile features a browser can see, deployed as a Chrome extension that scores any profile and reply thread in real time with a readable explanation of why.

Dataset

twitter_human_bots.csv from airt-ml (2023), hosted on Hugging Face under CC BY-SA 3.0. 37,438 X accounts labelled as human (25,013) or bot (12,425), giving a 2:1 class imbalance. The dataset downloads automatically on first run via the `datasets` library.

Raw fields:

Column	Type	Description
id	int64	Unique X user ID
screen_name	object	X handle
description	object	Profile bio text
location	object	Self-reported location
created_at	datetime	Account creation timestamp
followers_count	int64	Number of followers
friends_count	int64	Number of accounts followed
statuses_count	int64	Total tweets posted
favourites_count	int64	Total tweets liked
verified	bool	Verified status
default_profile	bool	Default profile theme
default_profile_image	bool	Default profile image
account_age_days	int64	Days since creation
average_tweets_per_day	float64	Mean tweets per day
account_type	object	Target label: human or bot

The dataset covers 2018 to 2020 and predates modern LLM-driven bot accounts. That is a real limitation for current-day performance and is discussed in the Limitations section.

Model Architecture and Training Pipeline

Feature Engineering

37 numeric features engineered from the raw fields, grouped into 8 categories:

Category	Count
Raw counts	6
Log transforms	6
Behavioural ratios	4
Profile completeness	8
Screen name patterns	3
Activity anomalies	2
Bio content signals	4
Account-type signals	4

The full ordered list is in `src/config.py` under `numeric_features`. All features are z-score normalised with a scaler fit on the training split only to avoid leakage.

The account-type signals category (`bio_has_news_keywords`, `bio_has_org_keywords`, `bio_likely_organisation`, `is_established_account`) exists because the airt-ml dataset labels accounts as human or bot but does not distinguish organisation accounts (news outlets, brands, verified institutions) from individual humans. Without these features, the model has no direct signal that an account looks like a publication or a company rather than a person.

Training Pipeline

1. Download the dataset from Hugging Face and cache to disk.
2. Clean raw fields, fill missing values, cast types.
3. Engineer the 37 features above.
4. Random 70/15/15 train/val/test split with a fixed seed.
5. Build a 50,000-word vocabulary from the training corpus for the GloVe-based text models.
6. Train BiGRU-LSTM, CNN-BiLSTM, and XGBoost with class-weighted loss to handle the 2:1 imbalance.
7. Tune the decision threshold per model on the validation set.
8. Evaluate on the held-out test set using F1, accuracy, precision, recall, ROC-AUC, and PR-AUC.
9. Save the best model by F1 and wire it into a FastAPI backend with `/predict` and `/predict_batch` endpoints.
10. Validate externally on MGTAB using the same XGBoost architecture.

Models

1. BiGRU-LSTM with gated fusion: a bidirectional GRU (Cho et al., 2014) followed by an LSTM (Hochreiter and Schmidhuber, 1997) over GloVe embeddings (Pennington, Socher and Manning, 2014) of the user's bio. The text branch combines with a `NumericNet` over the 37 numeric features through a learned sigmoid gate that decides per-sample which branch to weigh more.
2. CNN-BiLSTM with gated fusion: a multi-kernel 1D CNN in the style of Kim (2014), with kernel sizes 3, 4, 5 and max-over-time pooling, followed by a bidirectional LSTM. Same `NumericNet` and gated fusion.
3. XGBoost (Chen and Guestrin, 2016): gradient-boosted trees on the 37 engineered features. `scale_pos_weight = 2.03` for the 2:1 imbalance. Gradient-boosted trees are a strong default for tabular feature-based classification (Shwartz-Ziv and Armon, 2022; Grinsztajn, Oyallon and Varoquaux, 2022), which the results below confirm.

All three use class-weighted loss, early stopping on validation F1, and a per-model decision threshold tuned on the validation set.

Evaluation

Test Set Performance

The test set is 5,616 users which were held out before any modelling. Thresholds are tuned per model on the validation set and fixed before scoring the test set.

Model	F1	Accuracy	Precision	Recall	ROC-AUC	PR-AUC	Threshold	Train Time
XGBoost	0.8076	0.8725	0.8336	0.7832	0.9329	0.9031	0.580	1 s
BiGRU-LSTM	0.7914	0.8549	0.7777	0.8056	0.9220	0.8872	0.520	829 s
CNN-BiLSTM	0.7503	0.8292	0.7497	0.7509	0.9001	0.8486	0.570	49 s

XGBoost wins on F1 and accuracy. BiGRU-LSTM edges it on recall but drops on precision, so it catches slightly more bots at the cost of flagging more humans. Training time spans three orders of magnitude (1 second for XGBoost against 829 seconds for BiGRU-LSTM) with no accuracy return on the extra compute. The top features by gain are `verified` (0.20), `followers_count` (0.11), `is_established_account` (0.10), and `log_followers_count` (0.07). Full charts, ROC curves, confusion matrices, and training loss are in `notebooks/analysis.ipynb`.

Post-Deployment Benchmark on 50 Verified Organisations

The airt-ml dataset does not flag organisation accounts separately from regular human accounts, which makes orgs a blind spot the standard test set cannot catch. Early live testing of the extension confirmed the concern, where CNN scored 92/100 and Al Jazeera scored 84/100, which signalled that they were bot accounts when that is not the case.

I built a custom benchmark of 50 verified organisation accounts (15 news, 10 tech, 10 retail, 8 sports, 7 NGOs and government) to find the cause of the false flagging of accounts like CNN and Al Jazeera. If the model was broken, benchmark scores would match the live extension. If the model was fine, the bug had to be in the extension.

Given complete metadata, the model scores all 50 organisations as HUMAN, with no org scoring above 50/100. A secondary news-organisation override in the extension only shifts the score on 2 accounts (NFL, F1), and neither shift flips the classification. The benchmark clears the model and points at the DOM scraping path in `content.js` as the actual source of the live CNN and Al Jazeera errors. That is the next item in Future Work.

External Validation on MGTAB

MGTAB (Liu et al., 2023, arXiv:2301.01123) is a published bot detection dataset with 10,199 expert-annotated users and 788 pre-extracted features per user. I trained the same XGBoost architecture on MGTAB's features to see how the approach transfers.

Model	F1 on MGTAB
Random Forest (Liu et al., 2023)	~0.84
GCN (Liu et al., 2023)	~0.86
RGT (Liu et al., 2023)	~0.89
XGBoost (this project)	0.8364

The XGBoost approach lands at MGTAB's published Random Forest baseline. The graph-based models (GCN, RGT) pull ahead by 3 to 6 F1 points because they use the multi-relational graph, which a browser extension cannot see at inference. Implementation in `src/mgtab_eval.py`.

Ablation: Sentence-BERT Bio Embeddings

I tested whether richer bio representations help. I generated 384-dimensional Sentence-BERT embeddings (Reimers and Gurevych, 2019) for every bio in the dataset using `all-MiniLM-L6-v2` and concatenated them with the 37 numeric features for a fused XGBoost model.

Model	F1
XGBoost baseline (37 features)	0.8076
XGBoost + SBERT fused (421 features)	0.8051
Delta	-0.0025

The fused model regressed by 0.0025 F1 against the baseline. The dataset explains the result: 19% of bios are empty and the median non-empty bio is 53 characters, shorter than the input SBERT is trained on. There isn't enough signal in most bios for SBERT to contribute above what the existing bio features (`has_description`, `description_length`, `bio_word_count`, `bio_has_news_keywords`, `bio_has_org_keywords`) already extract. The deployed model stays at the 37-feature baseline. Implementation in `src/embed_bios.py`, with the fused training variant handled inside `src/train.py`.

How to Run

The backend is deployed at <https://mobucheeri-twitter-bot-detector.hf.space>. The extension calls this endpoint, so no local setup is needed.

1. The extension is pending review on the Chrome Web Store. If approved, it will be installable from there directly. However, it is not guaranteed that the Chrome Web Store will approve it, so this is the only method of deployment so far.
2. In the meantime, to install manually:
 - Clone the repo
 - Open `chrome://extensions` in Chrome
 - Turn on Developer mode
 - Click Load unpacked and select the `extension/` folder

Visit any profile on x.com to see the scoring panel. Visit any tweet thread to see per-reply heuristic flagging.

Development setup

To retrain the models, run the evaluations, or reproduce any of the work in this repo:

Requirements

- Python 3.11
- Google Chrome (any recent version)
- Roughly 4 GB free disk for the dataset, embeddings, and trained checkpoints

Setup

```
git clone https://github.com/mobucheeri-datasci/x-bot-detector
cd x-bot-detector
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Train and Evaluate

```
python src/data.py
python src/train.py
python src/orgs_eval.py
```

`train.py` auto-detects the SBERT bio embeddings cache at `data/processed/bio_embeddings.npy`. If it exists, `train.py` also trains the fused XGBoost variant as a fourth model. If not, it skips that step.

Optional: Reproduce the Sentence-BERT Ablation To include the SBERT fused model described under Evaluation, run `embed_bios.py` before `train.py`:

```
python src/embed_bios.py
python src/train.py
```

External Validation on MGTab

```
pip install gdown
mkdir -p data/raw/mgtab && cd data/raw/mgtab
gdown 1gbwN0oU1JB8RrTu2a5j9KMNVa9wX72Fe -O mgtab.zip && unzip mgtab.zip
cd ../../..
python src/mgtab_eval.py
```

Run the backend locally (optional) If you want to run the backend on your own machine instead of using the hosted Hugging Face Space:

```
uvicorn api.app:app --reload --port 8000
```

```
uvicorn api.app:app --reload --port 8000
```

Then change `apiBase` in `extension/background/service-worker.js` from the Hugging Face URL to `http://localhost:8000` before loading the extension.

Note

Trained checkpoints, the preprocessed dataset, the SBERT embedding cache, and MG TAB data are gitignored. The analysis notebook is committed with cell outputs preserved, so it renders on GitHub without re-running anything.

Repo Structure

```
twitter-bot-detector/
├── README.md
├── requirements.txt
├── .gitignore
├── data/
│   ├── raw/twitter_human_bots.csv
│   └── processed/processed.pt
├── models/checkpoints/
├── src/
│   ├── config.py
│   ├── data.py
│   ├── models.py
│   ├── train.py
│   ├── embed_bios.py
│   ├── orgs_eval.py
│   └── mgtab_eval.py
├── api/app.py
├── extension/
│   ├── manifest.json
│   ├── content/
│   ├── popup/
│   ├── background/
│   └── icons/
├── X Bot Detection_ Presentation.pdf
└── notebooks/analysis.ipynb
```

Limitations and Future Work

Limitations

The dataset is from 2018 to 2020 and contains no modern LLM-driven bot accounts. Performance on current-day ChatGPT-style bots is an open question.

The deployed model only sees profile-level features. It cannot use the social graph or a full tweet history, both of which are known to help. Graph-based models on MGTAB reach 0.86 to 0.89 F1 partly because they have access to the relational graph.

The Sentence-BERT bio embedding ablation regressed against the baseline (see Evaluation). Improving bio-text modelling for short or empty bios is an open question.

The live extension's only observed misclassifications (Al Jazeera at 84/100, CNN at 92/100) trace to incomplete DOM scraping in `content.js`, not to the model itself.

Future Work

1. Scraper fixes for the live extension. Recovering follower count and account age from X's hover cards would resolve the Al Jazeera and CNN errors.
2. Tweet content features. The current dataset has bios but no tweet histories.
3. Graph neural networks. GNNs sit a few F1 points above feature-only models on TwiBot-22 (Feng et al., 2022) and MGTAB, but the graph data is not available to a browser extension at inference time.
4. Temporal behavioural signals from visible tweet timestamps on a profile page.
5. Cross-platform generalisation to Instagram, Threads, Reddit, or YouTube.
6. Find and test more recent datasets that would more likely record LLM-driven bot behaviours across the features.

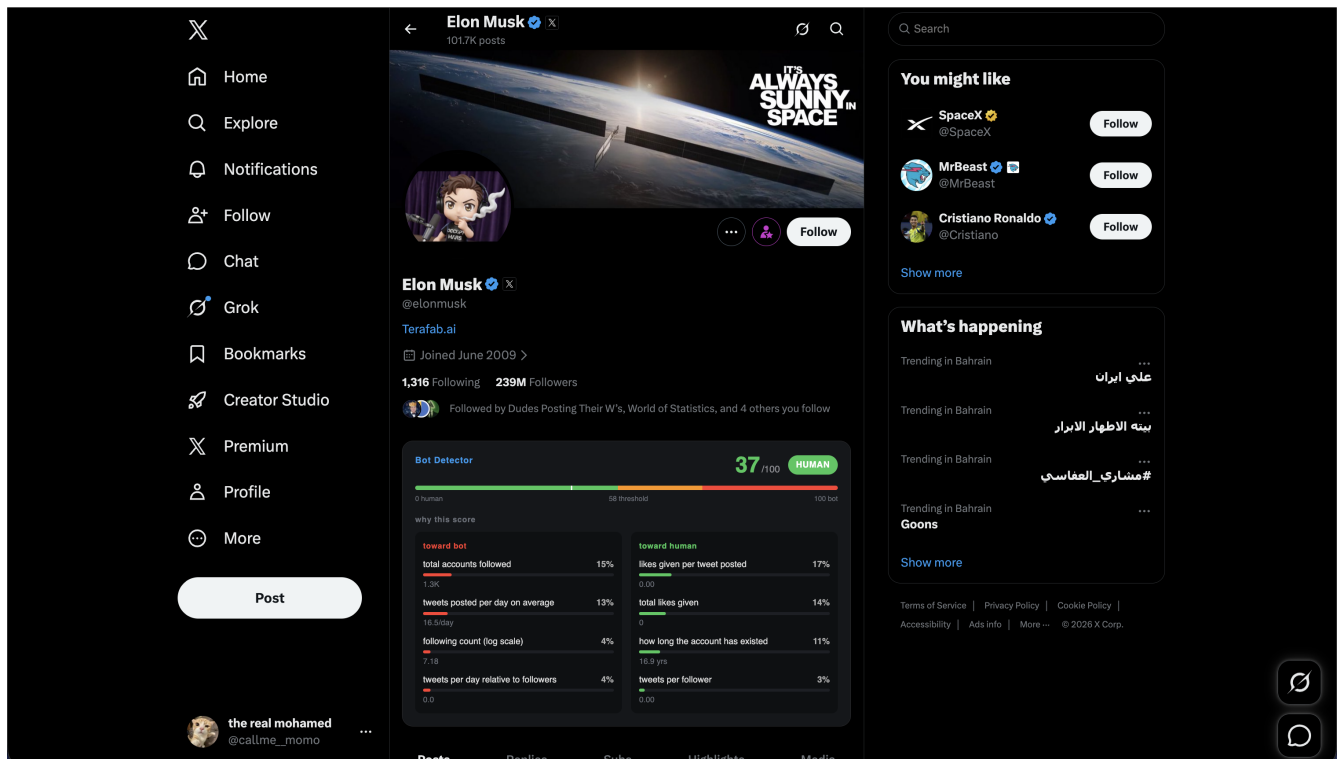
Screenshots and Demo

Demo Video

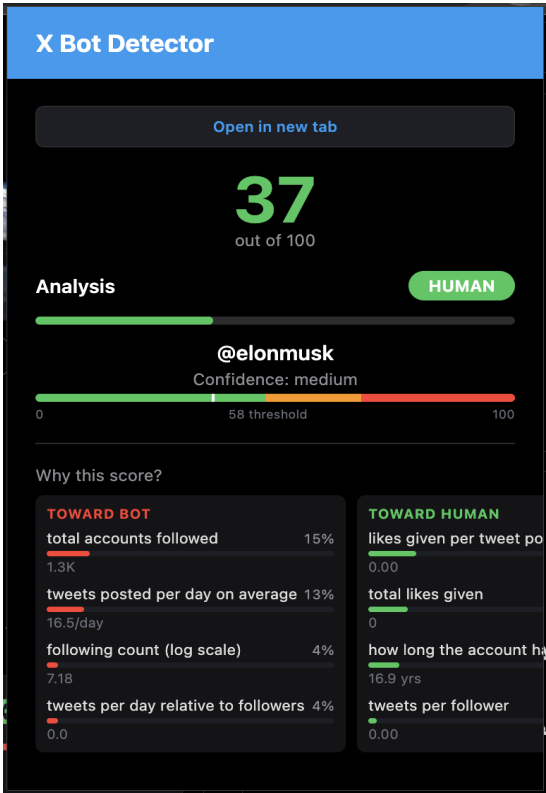
A 3-minute walkthrough of the extension in action: <https://www.loom.com/share/f40e6a858c684b91973baff2f7c30e11>

Screenshots

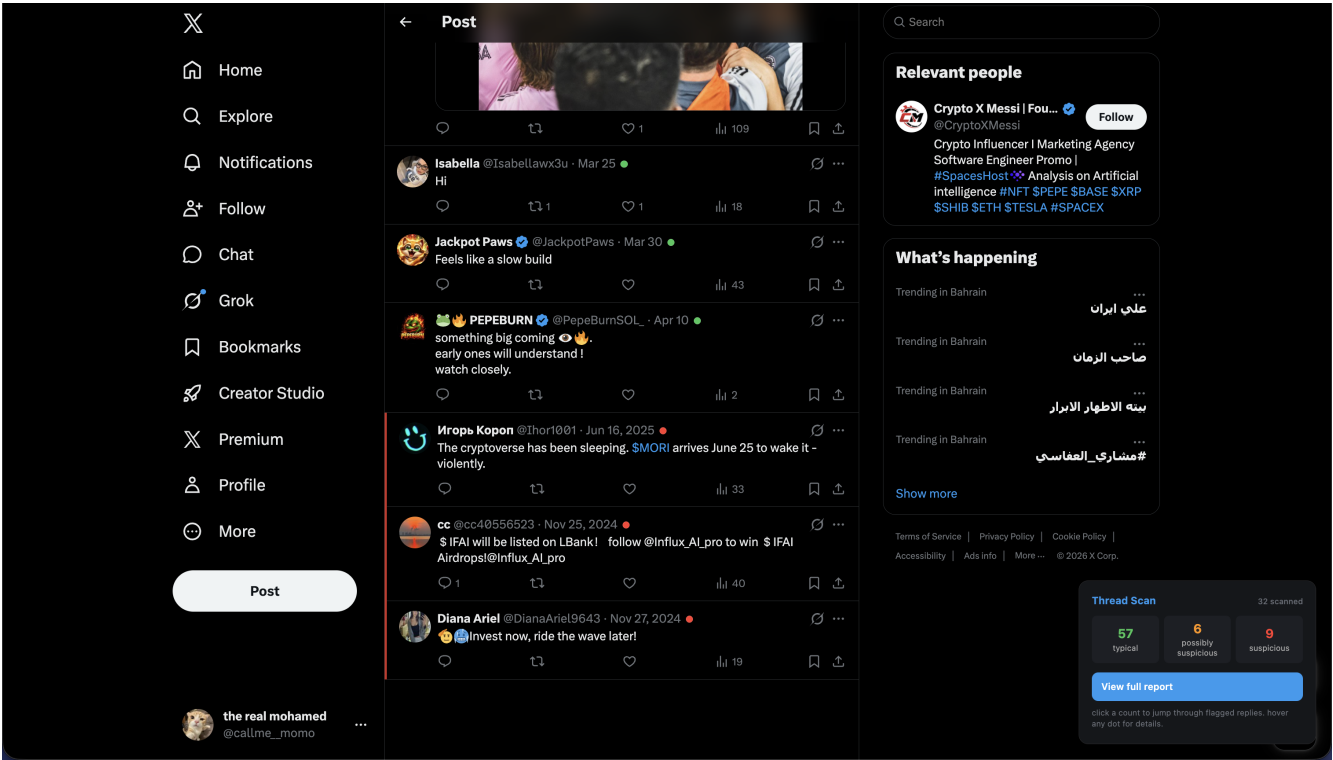
Profile panel on an X user profile, with per-feature contributions in percentages:



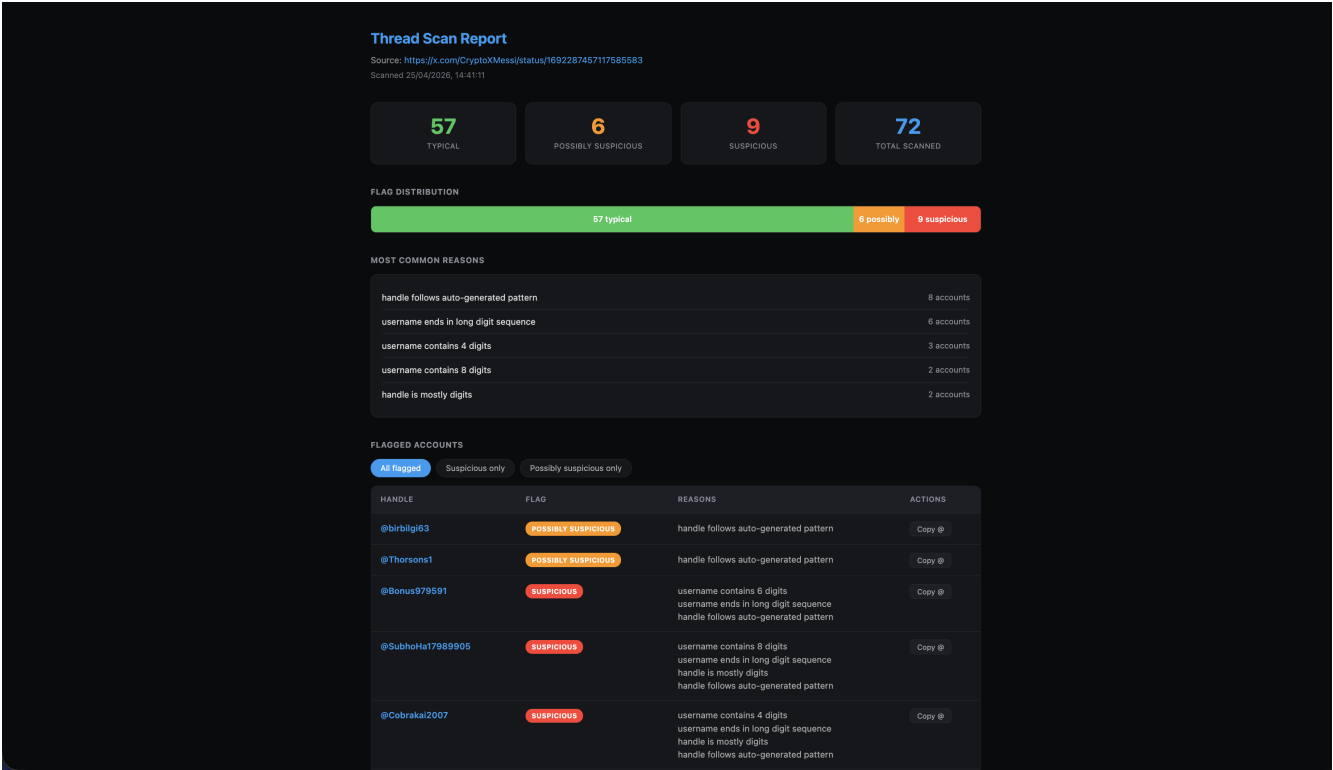
Toolbar popup with the same breakdown:



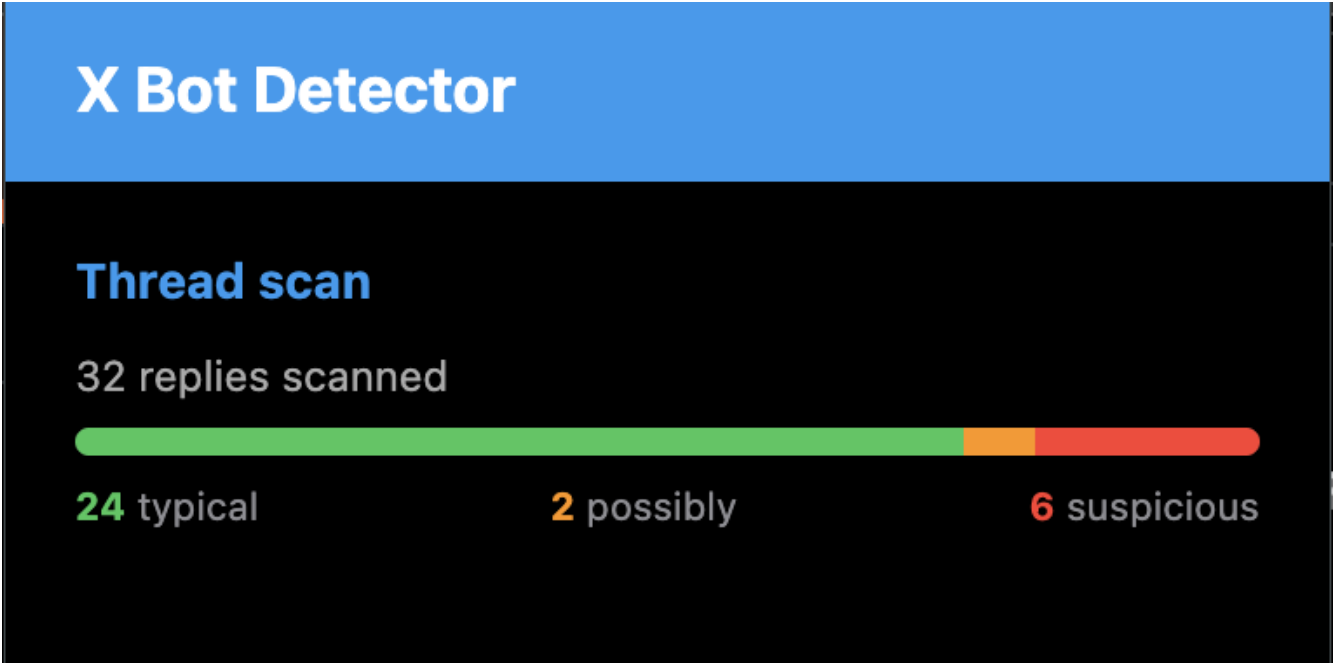
Thread scan on a reply section, with coloured dots next to flagged usernames, coloured left-borders on flagged replies, and a summary panel at the bottom right:



Full report view, opened in a new tab from the thread panel, with flag distribution, most common reasons, and table of flagged accounts:



Toolbar popup on a thread page, summarising the scan across all replies:



License and Acknowledgments

License

The training dataset (`twitter_human_bots.csv` from airt-ml) is distributed under CC BY-SA 3.0 and is not redistributed in this repository; it downloads from Hugging Face on first run. MGTab (Liu et al., 2023) is distributed under its own terms by the original authors.

Acknowledgments

- airt-ml for publishing the `twitter-human-bots` dataset on Hugging Face.
- Liu et al. (2023) for making the MGTab dataset and baselines public.
- The authors of the libraries used in this project: PyTorch, XGBoost, scikit-learn, pandas, NumPy, FastAPI, Hugging Face `datasets`, and Sentence-Transformers.

References

- airt-ml (2023). *Twitter Human Bots Dataset*. Hugging Face. Available at: <https://huggingface.co/datasets/airt-ml/twitter-human-bots> [Accessed April 2026].
- Bessi, A. and Ferrara, E. (2016). 'Social bots distort the 2016 U.S. presidential election online discussion'. *First Monday*, 21(11).
- Chen, T. and Guestrin, C. (2016). 'XGBoost: A scalable tree boosting system'. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785 to 794.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014). 'Learning phrase representations using RNN encoder-decoder for statistical machine translation'. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724 to 1734.
- Cresci, S. (2020). 'A decade of social bot detection'. *Communications of the ACM*, 63(10), pp. 72 to 83.
- Feng, S., Tan, Z., Wan, H., Wang, N., Chen, Z., Zhang, B., Zheng, Q., Zhang, W., Lei, Z., Yang, S., Feng, X. and Luo, M. (2022). 'TwiBot-22: Towards graph-based Twitter bot detection'. *Advances in Neural Information Processing Systems*, 35, pp. 35254 to 35269.
- Ferrara, E., Varol, O., Davis, C., Menczer, F. and Flammini, A. (2016). 'The rise of social bots'. *Communications of the ACM*, 59(7), pp. 96 to 104.
- Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022). 'Why do tree-based models still outperform deep learning on typical tabular data?'. *Advances in Neural Information Processing Systems*, 35, pp. 507 to 520.
- Hochreiter, S. and Schmidhuber, J. (1997). 'Long short-term memory'. *Neural Computation*, 9(8), pp. 1735 to 1780.
- Howard, P.N., Woolley, S. and Calo, R. (2018). 'Algorithms, bots, and political communications in the US 2016 election'. *Journal of Information Technology and Politics*, 15(2), pp. 81 to 93.
- Imperva (2024). *2024 Bad Bot Report*. Available at: <https://www.imperva.com/resources/resource-library/reports/2024-bad-bot-report/> [Accessed April 2026].
- Kim, Y. (2014). 'Convolutional neural networks for sentence classification'. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746 to 1751.
- Liu, C., Wang, T., Zhou, X., Liu, J., Yu, X., Zhao, Y. and Lin, X. (2023). 'MGTab: A multi-relational graph-based Twitter account detection benchmark'. *arXiv preprint*, arXiv:2301.01123.
- Pennington, J., Socher, R. and Manning, C. (2014). 'GloVe: Global vectors for word representation'. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532 to 1543.
- Reimers, N. and Gurevych, I. (2019). 'Sentence-BERT: Sentence embeddings using Siamese BERT-networks'. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982 to 3992.
- Shao, C., Ciampaglia, G.L., Varol, O., Yang, K.C., Flammini, A. and Menczer, F. (2018). 'The spread of low-credibility content by social bots'. *Nature Communications*, 9(1), p. 4787.
- Shwartz-Ziv, R. and Armon, A. (2022). 'Tabular data: Deep learning is not all you need'. *Information Fusion*, 81, pp. 84 to 90.